# C MIDTERM EXAM SURVIVAL GUIDE

Konsep Pemrograman (CS101) • High-Yield Topic Cheatsheet & Trap Breakdown

ISO C99/C11 STANDARD | 100% CALIBRATED TO 6 MIDTERM MODULES | MULTIPLE CHOICE TRAP DEFENSE | QUICK REFERENCE

## 1 Data Types & Variables (Module 1)

| Type | Bytes | Format | Value Range (32/64-bit OS) | Key Exam Behavior |
|---|---|---|---|---|
| `char` | 1 byte | `%c` / `%d` | -128 to 127 (or 0 to 255) | Stored as ASCII integer. Arithmetic with `'0'` converts digit char to int. |
| `int` | 4 bytes | `%d` / `%i` | -2,147,483,648 to 2,147,483,647 | Default integer type. Overflow wraps around on two's complement. |
| `unsigned int` | 4 bytes | `%u` | 0 to 4,294,967,295 | Cannot be negative. `0u - 1u` wraps around to maximum `4294967295`. |
| `float` | 4 bytes | `%f` | $\pm 3.4 \times 10^{38}$ (~6-7 dec digits) | Single precision. Float literals need suffix `f` (e.g. `3.14f`). |
| `double` | 8 bytes | `%f` / `%lf` | $\pm 1.7 \times 10^{308}$ (~15-17 dec digits) | Default for float literals (e.g. `3.14`). In `scanf`, MUST use `%lf`! |

### 💡 WHAT TO REMEMBER

- **Format Specifier Precision:** `%.2f` rounds output to 2 decimal places. `%5d` pads with spaces to width 5.
- **ASCII Math:** `'5' - '0' == 5`. `'A' + 32 == 'a'`. Characters are directly swappable with integer codes.
- **`const` Qualifier:** Declares read-only variables. Reassignment causes a compile error.
- **Scope:** Inner `{ ... }` blocks can declare variables with the same name, shadowing outer variables until the block closes.

### ⚠️ TRAPS TO WATCH OUT FOR (PROFESSOR FAVORITES)

- **Integer Division Truncation:**
  `double x = 5 / 2;` → Stores **2.0**, NOT 2.5!
  *Why:* Both `5` and `2` are ints, so `5/2` truncates to int `2` before assigning.
- **Casting Order:**
  `(double)(5 / 2) == 2.0` (parentheses divide first!)
  `(double)5 / 2 == 2.5` (5 becomes 5.0 first!)
- **Float Literal Equality:**
  `float f = 0.1f; if (f == 0.1)` → **FALSE!**
  *Why:* `0.1` is a double. 0.1 has no exact binary representation, so 32-bit float and 64-bit double differ slightly!

## 2 Operators & Expressions (Module 2)

| Prec | Operators | Description | Associativity | Sneaky Exam Trap |
|---|---|---|---|---|
| **1 (High)** | `() [] -> . x++ x--` | Postfix, array subscript, parens | Left-to-Right | `x++` returns OLD value first! |
| **2** | `++x --x + - ! ~ (type) sizeof` | Unary operators, prefix inc/dec, cast | **Right-to-Left** | `sizeof` doesn't evaluate side-effects! |
| **3** | `* / %` | Multiplicative (multiply, divide, mod) | Left-to-Right | `%` only valid on integers! |
| **4** | `+ -` | Additive (binary addition, subtraction) | Left-to-Right | Higher precedence than shifts! |
| **5** | `<< >>` | Bitwise left & right shifts | Left-to-Right | `1 << 2 + 3` is `1 << (2+3) = 32`! |
| **6** | `< <= > >=` | Relational comparisons | Left-to-Right | `1 < x < 10` does NOT check range! |
| **7** | `== !=` | Equality checks | Left-to-Right | Don't confuse with assignment `=`! |
| **8–10** | `&` then `^` then `\|` | Bitwise AND, XOR, OR | Left-to-Right | Lower precedence than `==`! |
| **11** | `&&` | Logical AND | Left-to-Right | **Short-circuits:** stops if left is 0! |
| **12** | `\|\|` | Logical OR | Left-to-Right | **Short-circuits:** stops if left is 1! |
| **13** | `?:` | Ternary conditional | **Right-to-Left** | `a ? b : c ? d : e` groups right-to-left. |
| **14 (Low)** | `= += -= *= /= %=` | Assignment & augmented assignment | **Right-to-Left** | `x *= y + 1` means `x = x * (y + 1)`! |

### 💡 CORE MECHANICS TO REMEMBER

- **Pre vs Post Increment:**
  `int a = 5; int b = a++;` → `b = 5, a = 6` (old value copied).
  `int a = 5; int b = ++a;` → `b = 6, a = 6` (incremented first).
- **Modulo Sign Rule:** In C99, `a % b` ALWAYS takes the sign of the LEFT operand `a`:
  `-7 % 3 == -1` • `7 % -3 == 1` • `-7 % -3 == -1`.

### ⚠️ SHORT-CIRCUIT & COMPOUND TRAPS

- **Short-Circuit Skipping Side Effects:**

```
int a = 0, b = 5;
if (a && ++b) { }
printf("%d", b); // PRINTS 5, NOT 6! (++b never ran!)
```

- **Compound Assignment Right-Hand Precedence:**
  `x *= y + 3;` is parsed as `x = x * (y + 3)`, NOT `(x * y) + 3`!
- **Chained Comparisons Myth:**
  `if (5 < x < 2)` evaluates `(5 < x)` to `0` or `1`, and then compares `0 < 2` (always true)!

## 3 Conditionals: if, else & switch (Module 3)

### 💡 HOW CONDITIONALS WORK IN C

- **Boolean Truth Value:** In C, there is no separate primitive bool type in standard C90. **Any non-zero value is TRUE** (e.g. `1`, `-42`, `999`). Exactly `0` **is FALSE**.
- **Switch Structure:** Evaluates an integer or character expression once, then jumps to the matching `case` label.
- **Case Label Requirements:** Each `case` MUST be a compile-time constant integer or char (e.g. `case 1:`, `case 'A':`). You CANNOT use variables like `case x:`!
- **Ternary Operator:** `condition ? expr_if_true : expr_if_false` returns an expression value.

### ⚠️ EXAM TRAPS: 4 CLASSIC SNEAKY BUGS

- **1. The Assignment in Condition Trap:**
  `if (x = 5) { ... }` → **ALWAYS TRUE!**
  *Why:* Uses single `=` (assignment), NOT `==`. Sets `x = 5`, which evaluates to 5 (non-zero = true).
- **2. The Accidental Semicolon:**
  `if (score > 90); { printf("Passed"); }`
  *Why:* The semicolon immediately ends the `if`! The block following it runs UNCONDITIONALLY!
- **3. Dangling Else Binding:**
  In C, an `else` always binds to the **closest preceding unmatched** `if`, regardless of code indentation!
- **4. Switch Fallthrough without Break:**
  Once a `case` matches, ALL subsequent cases execute sequentially until a `break;` is encountered!

**Dangling Else Reality Check:**

```
int a = 1, b = 0;
if (a > 0)
    if (b > 0)
        printf("A");
else // <-- BINDS TO if (b > 0), NOT if (a > 0)!
    printf("B");
// OUTPUT: "B" is printed!
```

**Switch Fallthrough Reality Check:**

```
int x = 2;
switch(x) {
    case 1: printf("1");
    case 2: printf("2"); // Matches here!
    case 3: printf("3"); // Falls through!
    default: printf("D"); // Falls through!
}
// OUTPUT: Prints "23D" because breaks are missing!
```

## 4 Loops & Iteration (Module 4)

| Loop Type | Syntax | Condition Check | Min Iterations | Typical Use Case |
|---|---|---|---|---|
| `for` loop | `for (init; cond; step)` | Pre-tested (before body) | **0** times | Known iteration count, array traversal. |
| `while` loop | `while (cond) { ... }` | Pre-tested (before body) | **0** times | Event-driven, unknown loop count. |
| `do-while` loop | `do { ... } while (cond);` | **Post-tested** (after body) | **1 time (Guaranteed)** | Menu prompts, user input validation. |

### 💡 BREAK vs CONTINUE MECHANICS

- `break;`: Terminates the innermost loop or switch statement immediately. Execution jumps to the statement following the loop.
- `continue;`: Skips the remaining code in the *current* iteration:
  - In `for`: Jumps straight to the **step expression** (e.g. `i++`), then evaluates condition.
  - In `while` / `do-while`: Jumps directly to the **condition test**.
- **Do-While Semicolon:** The `do { ... } while (cond);` loop MUST end with a semicolon `;`.

### ⚠️ THE 4 DEADLIEST LOOP TRAPS

- **1. The Empty Body Semicolon:**
  `for (int i = 0; i < 5; i++);` → The loop does nothing 5 times! Semicolon acts as empty body. Block after runs once with `i = 5`!
- **2. While Loop Continue Freeze:**
  If you call `continue` before incrementing `i++` in a `while` loop, `i` never changes → **INFINITE LOOP!**
- **3. Unsigned Backward Loop:**
  `for (unsigned int i = 3; i >= 0; i--)`
  When `i = 0` decrements, it wraps to `4294967295` >= 0 → **NEVER TERMINATES!**
- **4. Post-Increment in Test:**
  `while (i++ < 3)` tests `i < 3`, but increments `i` regardless of true/false!

## 5 Functions & Procedures (Module 5)

### 💡 FUNCTION CONCEPTS & DEFINITIONS

- **Prototype (Declaration):** `int add(int a, int b);` informs the compiler about return type and parameters before `main()`.
- **Definition:** Contains the actual body of the function.
- **Function vs Procedure:** In C, a "procedure" is simply a function with return type `void` (does not return a value).
- **Scope of Local Variables:** Variables declared inside a function exist on the stack and are destroyed when the function returns.

### ⚠️ THE #1 EXAM CONCEPT: PASS-BY-VALUE!

- **C is STRICTLY Pass-by-Value:**
  When arguments are passed, C copies their values into parameter variables. Modifications inside the callee do NOT affect the caller!

```
void increment(int x) { x = x + 1; }
int main() {
    int val = 10;
    increment(val);
    printf("%d", val); // PRINTS 10, NOT 11!
}
```

### ⚠️ STATIC LOCAL VARIABLE RETENTION

A `static` local variable inside a function is **initialized only ONCE** and **retains its value between function calls**!

```
void counter() {
    static int c = 0; // Initialized once!
    c++;
    printf("%d ", c);
}
int main() {
    counter(); counter(); counter();
    // OUTPUT: 1 2 3 (NOT 1 1 1!)
}
```

### 💡 HOW TO TRACE RECURSION ON EXAMS

Always draw an activation tree downward, then return values upward:

```
int f(int n) {
    if (n <= 1) return 1; // Base case
    return n * f(n - 1);
}
// Trace f(3):
// f(3) = 3 * f(2)
//   f(2) = 2 * f(1)
//     f(1) = 1 (Base case reached!)
//   f(2) = 2 * 1 = 2
// f(3) = 3 * 2 = 6
```

*Watch out: Missing or unreachable base case causes Stack Overflow crash.*

## 6 Arrays & Strings (Module 6)

| Declaration | Memory Content | Valid Indices | Critical Exam Pitfall |
|---|---|---|---|
| `int a[5];` | Garbage values (uninitialized) | `0, 1, 2, 3, 4` | Accessing `a[5]` is **out of bounds** (undefined behavior)! |
| `int a[5] = {1, 2};` | `{1, 2, 0, 0, 0}` | `0, 1, 2, 3, 4` | **Partial Init Zero-Padding:** Unspecified elements become 0! |
| `int m[2][3];` | 2 rows, 3 cols (6 elements) | `[0..1][0..2]` | Stored in **Row-Major Order** contiguously in memory. |
| `char s[6] = "hello";` | `{'h','e','l','l','o','\0'}` | `0..5` (Length 5) | Requires **6 bytes** to store the trailing null terminator `\0`! |
| `char s[5] = "hello";` | `{'h','e','l','l','o'}` | **NO NULL BYTE!** | `printf("%s", s)` will over-read memory until a random `\0` is found! |

### 💡 SIZEOF vs STRLEN IN STRINGS

- `sizeof(str)` : Returns the **total buffer size in bytes** allocated to the array, regardless of string length.
- `strlen(str)` : Counts the characters up to (but not including) the first null terminator `'\0'`.
- *Example:*

```
char s[20] = "cat";
sizeof(s); // == 20 (array capacity)
strlen(s); // == 3  ('c', 'a', 't')
```

### ⚠️ 2D ARRAY INDEXING & ROW-MAJOR ORDER

- **Row vs Column:** In `int arr[R][C]`, `R` is the number of rows, `C` is the number of columns. Element is `arr[row][col]`.
- **Memory Layout:** Row 0 is stored first, immediately followed by Row 1 in linear memory:

```
int m[2][2] = {{10, 20}, {30, 40}};
// In memory: [10][20][30][40]
// m[0][0]=10, m[0][1]=20, m[1][0]=30, m[1][1]=40
```

### 🏆 10 GOLDEN RULES TO MEMORIZE BEFORE ENTERING THE EXAM ROOM

1. **5 / 2 is 2, NOT 2.5:** Dividing two integer literals always truncates fractional digits toward zero.
2. **Single '=' is assignment:** `if (x = 5)` sets `x = 5` and evaluates to TRUE. Always check for double `==`.
3. **Semicolons kill branches and loops:** Watch out for `if (...);`, `for (...);`, and `while (...);`. They create empty bodies.
4. **Switch falls through:** Without a `break;` statement, execution continues into subsequent cases regardless of their labels.
5. **Logical operators short-circuit:** In `0 && f()` and `1 || f()`, the right-hand function or expression is NEVER evaluated.
6. **Post-increment returns the old value:** `int y = x++;` gives `y` the value of `x` *before* it was incremented.
7. **C is strictly pass-by-value:** Changes to ordinary parameters inside functions do not modify the caller's variables.
8. **Static local variables remember:** `static int x = 0;` retains its value across function calls and initializes only once.
9. **Array bounds start at 0:** An array of size 5 has valid elements `arr[0]` through `arr[4]`. `arr[5]` is illegal!
10. **Strings must have room for '\0':** A 5-letter word requires an array of size at least 6 bytes to store the null terminator `\0`.